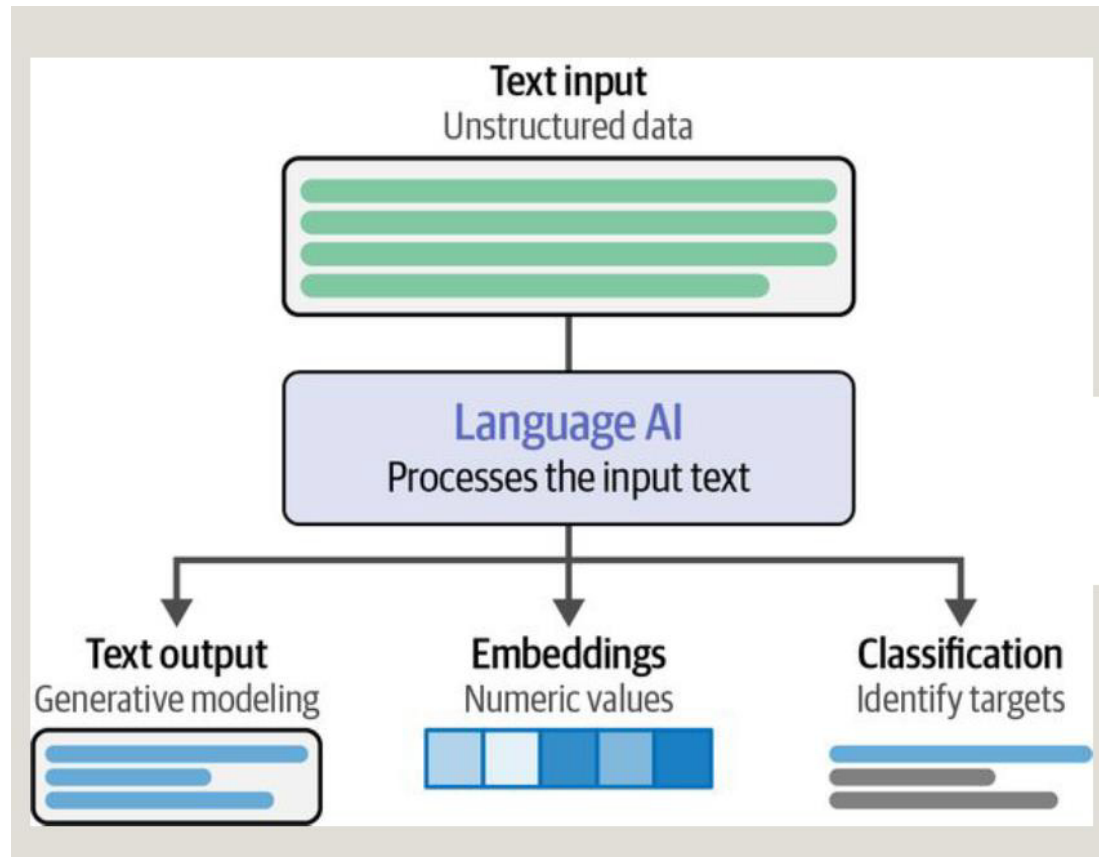
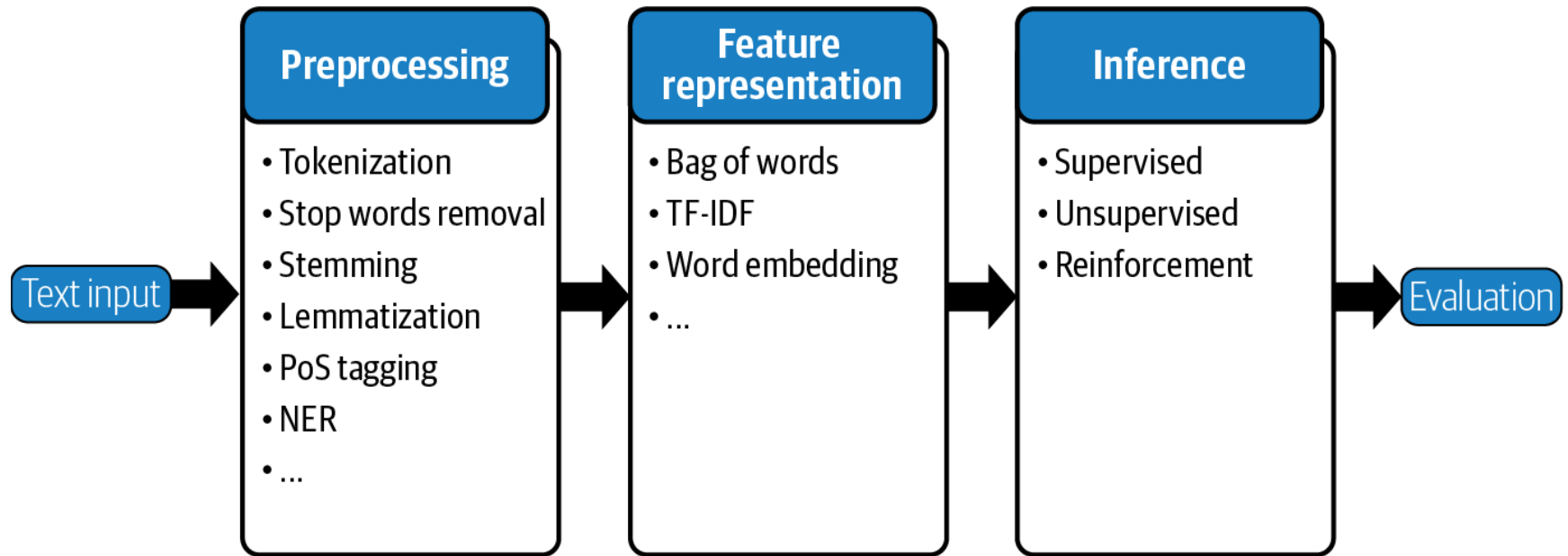


NLP: Text Preprocessing

NLP: Basic Flow Diagram



NLP Pipeline



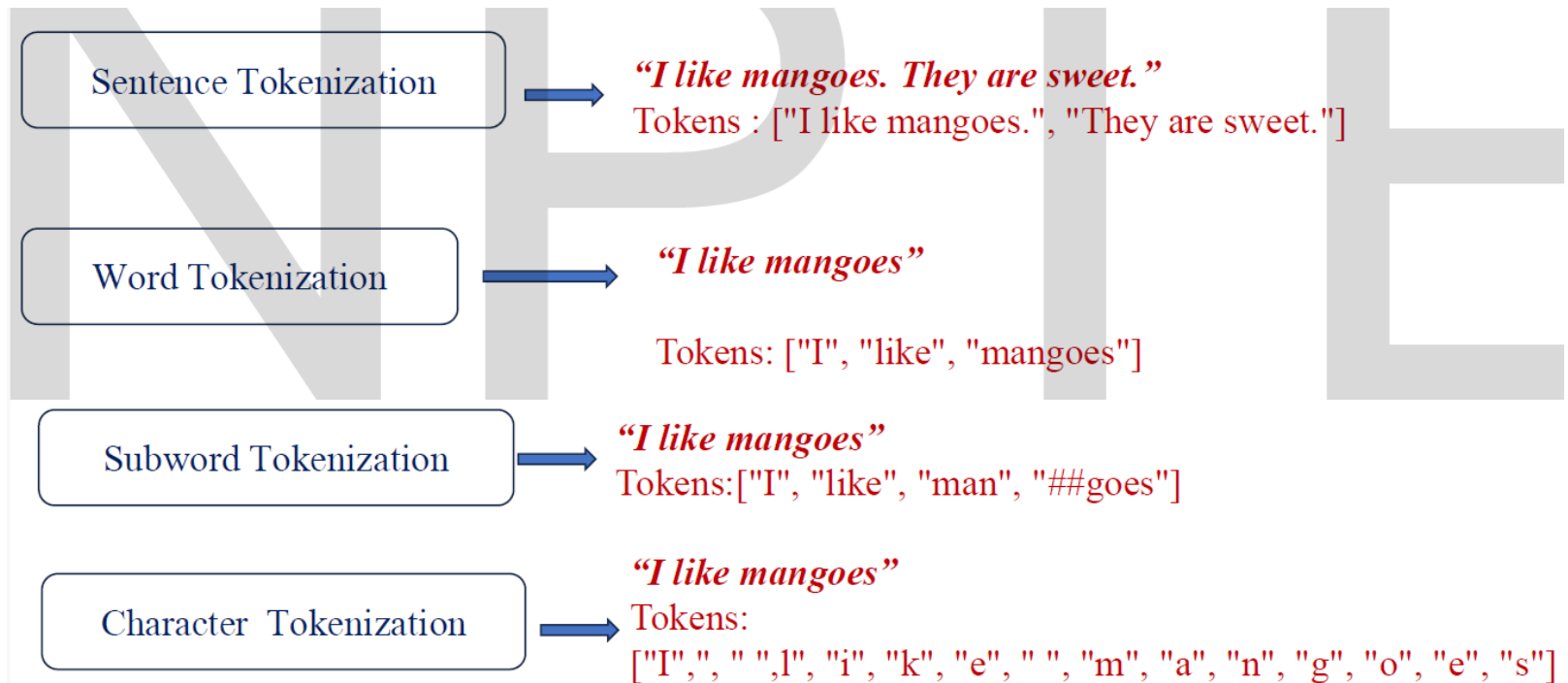
- **Text input data:**
 - Unstructured
 - Free-form and human-generated
 - Varies in length, style, grammar, and vocabulary
 - Often noisy (typos, emojis, abbreviations)
- **Text preprocessing** is an essential step in NLP that involves cleaning and transforming **unstructured text data** to prepare it for analysis.
- In order for the preprocessed information to be conveyed to the statistical inference algorithm, the tokens need to be translated into **predictive features**.
- A model is used to embed raw text into a **vector space**.
- Infer a decision in order to be actionable
- Unsupervised : Latent Semantic Analysis (LSA)
- Supervised: Recurrent Neural Networks (RNN) and variants

NLP: Python Packages, Libraries

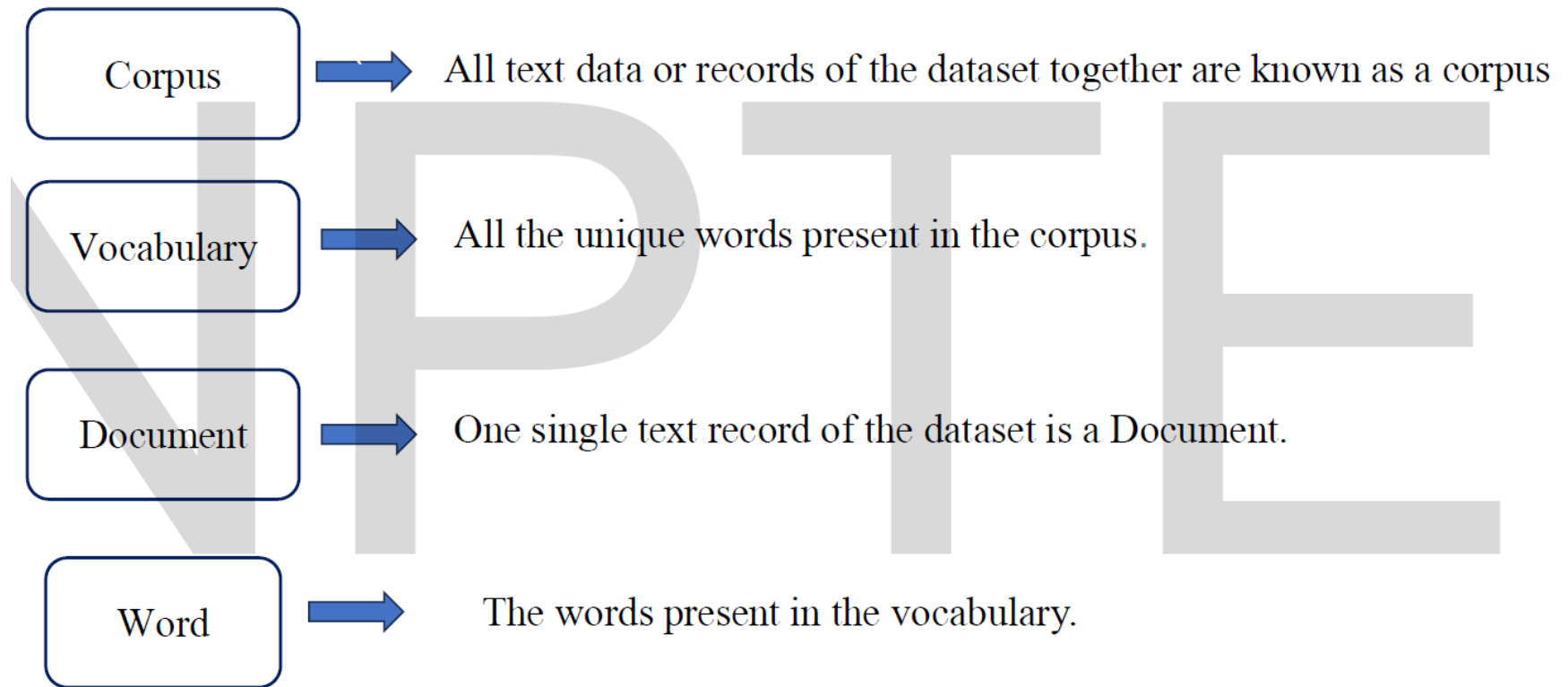
- **NLTK** (Natural Language Toolkits) - the most famous Python NLP library
- NLTK Data package includes a pretrained tokenizer *punkt* for English, which can be downloaded as well:
 - `import nltk`
 - `import nltk.data`
 - `nltk.download('punkt')`
- **TextBlob** is built on top of NLTK.
 - simplifies complex text analysis tasks—such as sentiment analysis, part-of-speech tagging, and translation
- **spaCy** is an NLP library designed to be fast, streamlined, and production-ready.
 - present only one algorithm (the best one) for each purpose.
- There are a few other libraries, such as **genism** (Generate Similar), that designed specifically for **unsupervised topic modeling**, document indexing, and semantic similarity retrieval.

Text Pre-processing: Tokenization

- Tokenization is a fundamental process in Natural Language Processing (NLP) that involves breaking down a stream of text into smaller units called tokens.
- A set of predetermined rules allows us to effectively convert a sentence into a list of tokens.



Tokenization: Key Terminology



Tokenization: Key Terminology

- *they lay back on the San Francisco grass and looked at the stars and their*
 - **Token**: an instance of that type in running text.
 - **Type**: an element of the vocabulary.
 - How many?
 - 15 tokens
 - 13 types

How many words in a corpus?

N = number of tokens

V = vocabulary = set of types, $|V|$ is size of vocabulary

Heaps Law = Herdan's Law = $|V| = kN^\beta$ where often $.67 < \beta < .75$

i.e., vocabulary size grows with $>$ square root of the number of word tokens

Corpus	Tokens = N	Types = $ V $
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
COCA	440 million	2 million
Google N-grams	1 trillion	13+ million

Tokenization: Whitespace tokenization

- Simplest tokenization method that splits text into tokens wherever white space characters (spaces, tabs, newlines) occur.

Tokens are implied to be *words*

Example:

Input text: students opened their books

Input token IDs: 11 298 34 567

Whitespace tokenizer issues

- *conjunctions*: isn't $\Rightarrow$ is, n't
- *hyphenated phrases*: prize-winning $\Rightarrow$ prize, -, winning
- *punctuation*: great movie! $\Rightarrow$ great, movie, !

(Word tokenizers require lots of specialized rules about how to handle specific inputs)

Tokenization: Whitespace tokenization

```
from nltk.tokenize import WhitespaceTokenizer
```

```
text = "Hello, world! This is NLP."
```

```
tokenizer = WhitespaceTokenizer()
```

```
tokens = tokenizer.tokenize(text)
```

```
print(tokens)
```

```
# Output: ['Hello,', 'world!', 'This', 'is', 'NLP.']
```

Whitespace tokenization: Major Issues

1. Multi-word Expressions Split Incorrectly

Proper nouns and compound terms get fragmented:

- "New York University" → ['New', 'York', 'University'] (loses the entity meaning)
- "San Francisco", "Los Angeles", "ice cream" all split into separate tokens

2. Punctuation Attached to Words

Punctuation remains stuck to adjacent tokens:

- "Hello, world!" → ['Hello,', 'world!'] instead of ['Hello', ',', 'world', '!']
- This complicates downstream processing like sentiment analysis or named entity recognition (NER)

3. Languages Without Spaces

- Completely fails for languages like Chinese, Japanese, or Korean where words aren't separated by spaces.

4. Special Constructs Broken

- Email addresses: john.doe@email.com might be split incorrectly
- URLs, phone numbers (800) 234-2333, and dates Mar 11, 1983 get fragmented
- Hyphenated words like "state-of-the-art" become separate tokens

5. Retrieval Problems

- Searching for "York University" might incorrectly match documents containing "New York University" because both contain "York" and "University" as separate tokens

Whitespace tokenization: Solutions and Better Approaches

- **Subword Tokenization (Byte Pair Encoding - BPE, WordPiece, SentencePiece)**
- Modern NLP uses subword methods that:
 - Break rare words into meaningful subword units
 - Handle compound words better
 - Manage out-of-vocabulary (OOV) words effectively
 - OOV in NLP means a word that a computer model cannot recognize because it was not in the text list used to train the model.

Subword Tokenization

- Subwords are expected to be meaningful units
- **Subwords** can be arbitrary substrings but subwords can be meaning-bearing units like the morphemes (parts of words)
 - A **morpheme** is the smallest meaning-bearing unit of a language
 - **Morphology** is the study of the way words are built up from morphemes
 - for example, the word fox consists of one morpheme (the morpheme fox) while the word cats consists of two: the morpheme cat and the morpheme -s that indicates plural.
 - “unlikeliest” has the morphemes {un-, likely, -est}
 - Here’s a sentence in English segmented into morphemes with hyphens:
 - He work-ed care-ful-ly wash-ing the glass-es
 - **Word forms** are the variations of a word that express different grammatical categories (tense, case, number, gender, etc.) and thus help convey the specific meaning and function of the word in a sentence
- **Unseen word can thus be represented by some sequence of known subword units. e.g. lower as {low, -er}**

Subword Tokenization: Byte-Pair-Encoding (BPE)

- **Main idea:** Use data to automatically tell what the tokens should be
 - **Token learner (Training Part)**
 - Raw train corpus $\Rightarrow$ Vocabulary (a set of tokens)
 - **Token segmenter (Testing Part)**
 - Raw sentences $\Rightarrow$ Tokens in the vocabulary

Subword Tokenization: Byte-Pair-Encoding (BPE)

- **Token learner: (Training Part)**
- *Raw train corpus $\Rightarrow$ Vocabulary (a set of tokens)*
 - Pre-tokenize the corpus in words & append a special end-of-word symbol _ to each word
 - Initialize vocabulary with the set of all individual characters
 - Choose 2 tokens that are most frequently adjacent (“A”, “B”)
 - Respect word boundaries
 - Add a new merged symbol (“AB”) to the vocabulary
 - Change the occurrence of the 2 selected tokens with the new merged token in the corpus
 - Continues doing this until k merges are done
- All k new symbols and initial characters are the final vocabulary
- ***What’s k? Open research question***

Subword Tokenization: Byte-Pair-Encoding (BPE)

- Example:

corpus

Freq. 5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_ , d , e , i , l , n , o , r , s , t , w

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_ , d , e , i , l , n , o , r , s , t , w , e r

Subword Tokenization: Byte-Pair-Encoding (BPE)

- Example:

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

Subword Tokenization: Byte-Pair-Encoding (BPE)

- Example:

merge	current vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

```
 $V \leftarrow$  all unique characters in  $C$            # initial set of tokens is characters
for  $i = 1$  to  $k$  do                           # merge tokens  $k$  times
     $t_L, t_R \leftarrow$  Most frequent pair of adjacent tokens in  $C$ 
     $t_{NEW} \leftarrow t_L + t_R$                  # make new token by concatenating
     $V \leftarrow V + t_{NEW}$                        # update the vocabulary
    Replace each occurrence of  $t_L, t_R$  in  $C$  with  $t_{NEW}$  # and update the corpus
return  $V$ 
```

Figure 2.6 The training part of the BPE algorithm for taking a corpus broken up into individual characters or bytes, and learning a vocabulary by iteratively merging tokens. Figure adapted from [Bostrom and Durrett \(2020\)](#).

Subword Tokenization: Byte-Pair-Encoding (BPE)

- Token segmenter: (Testing Part)
 - Just runs on the test data the merges we have learned from the training data, greedily, in the order we learned them
 - First we segment each test sentence word into characters
 - Then we apply the first merge rule
 - e.g., replace every instance of “e”, “r” in the test corpus with “er”
 - Then the second merge rule
 - e.g., replace every instance of “er”, “_” in the test corpus with “er_” and so on

Subword Tokenizers in Models

Model	Tokenizer	Vocabulary Size
BERT base (uncased) [2018]	WordPiece	30,522
BERT base (cased) [2018]	WordPiece	28,996
GPT-2 [2019]	BPE	50,257
Flan-T5 [2022]	SentencePiece	32,100
GPT-4 [2023]	BPE	> 100,000
StarCoder2 [2024]	BPE	49,152
Llama2 [2023]	BPE	32,000

You can play with different tokenizers here: <https://tiktokenizer.vercel.app/>

Tokenization: Practical Recommendation

- For production NLP systems, use established libraries like:
- **spaCy**: Handles punctuation, contractions, and multi-word expressions intelligently
- **NLTK's advanced tokenizers**: Beyond `WhitespaceTokenizer`, use `word_tokenize()` which uses Punkt sentence tokenizer
- **Hugging Face tokenizers**: BPE/WordPiece implementations for transformer models